

Conversão JSON para Protocolo Allexo BLE

Sumário

Conversão JSON para Protocolo Allexo BLE.....	1
Protocolo Allexo BLE:	2
Conversão comando Status:	3
Comando de leitura:	3
JSON Atual:	3
Requisição (Central para o equipamento)	3
Resposta (Equipamento para o Central)	3
Comando de escrita:	4
Conversão comando RTC:	4
Comando de Leitura:	4
JSON Atual:	4
Requisição (Central para o equipamento)	4
Resposta (Equipamento para o Central)	4
Comando de escrita:	5
JSON Atual:	5
Escrita (Central para o equipamento)	5
Resposta (Equipamento para o Central)	5
Configuração dos Drivers:	6
Formatação exigida para o campo Payload:	6

Abaixo, segue a conversão dos comandos JSON referenciados no documento “Especificação de Firmware - Rev A” para um protocolo serial binário, otimizado para o processamento direto no firmware via Bluetooth Low Energy (BLE) entre o aplicativo (Central) e o equipamento (ALX).

Regras Gerais do Protocolo:

- **Modelo Pergunta/Resposta:** Todo comando enviado obrigatoriamente exige uma resposta do dispositivo receptor.
- **Comandos de Leitura (Requisição):** Não possuem dados a serem gravados. Nestes casos, o campo 'Payload' é suprimido e o 'Tamanho do Payload' recebe o valor 0x00.
- **Controle de Fluxo:** O byte de 'Direção' indica a origem da instrução (Central para o ALX ou ALX para o Central).
- **Validação de Integridade:** O frame é validado utilizando o algoritmo CRC-16/Kermit (Init: 0x0000).

Exemplos Abordados:

Para ilustrar o funcionamento, foram convertidos dois comandos distintos:

1. **Status:** Exemplo de comando unicamente de requisição (não possui estrutura de configuração).
2. **Conexão:** Exemplo de comando híbrido, suportando rotinas de leitura e escrita.

Nota: Esta lógica de conversão é o padrão para todo o sistema. A única exceção é o comando de envio de drivers, que possui uma estrutura de dados particular e é detalhado em um tópico específico deste documento.

Protocolo Alexo BLE:

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	Payload (X Byte)	CRC 16 (2 Bytes)
0xA5	0x00: Central -> ALX 0x01 – ALX -> Central	0XXXXX	0XX	0XXXXX		0XXXXX

Conversão comando Status:

Comando de leitura:

JSON Atual:

```
19  {
20      "sts": {
21          "serial": "00000110",
22          "model": "ALX-5DLG-01",
23          "mac": "D8132AB65070",
24          "frmw": "0.0.1",
25          "rtc": 1785185292,
26          "ram_p": 56,
27          "flash_p": 32,
28          "bat_v": 3.44,
29          "bat_p": 35,
30          "reg_fst": 1785185292,
31          "reg_lst": 1785185292
32      }
33 }
```

Requisição (Central para o equipamento)

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	CRC 16 (2 Bytes)
0xA5	0x00: Central -> ALX	0x01	0x00	0x00	0xD5B2

Resposta (Equipamento para o Central)

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	Payload (X Byte)	CRC 16 (2 Bytes)
0xA5	0x01: ALX -> Central	0x01	0x00	0x42	char serial [10] char model [15] char mac [15] char rev [8] uint32_t timestamp uint8_t ext_mem_use float bat_voltage uint8_t bat_voltage_percent uint32_t reg_fst_timestamp uint32_t reg_lst_timestamp	0XXXXX

Comando de escrita:

O comando de status do dispositivo não possui requisição de escrita.

Conversão comando RTC:

Comando de Leitura:

JSON Atual:

1.5. Requisição da programação do RTC interno

Pergunta

```
{
  "cmd": "prg_rtc_r",
  "pmt": null
}
```

Resposta

```
{
  "cmd": "prg_rtc_r",
  "sts": {
    "timestamp": 1785185292,
    "utc": 0,
    "ntp": true
  }
}
```

Requisição (Central para o equipamento)

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	CRC 16 (2 Bytes)
0xA5	0x00: Central -> ALX	0x03	0x00	0x00	<u>0xE602</u>

Resposta (Equipamento para o Central)

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	Payload (X Byte)	CRC 16 (2 Bytes)
0xA5	0x01: ALX -> Central	0x03	0x00	0x06	UInt32_t timestamp UInt8_t utc UInt8_t ntp	0XXXXX

Comando de escrita:

JSON Atual:

1.6. Programação do RTC interno

Comando

```
{  
  "cmd": "prg_rtc_w",  
  "pmt": {  
    "timestamp": null, (null indica que não ajusta manualmente o rtc)  
    "utc": 0,  
    "ntp": true (true = a cada conexão com a internet busca horário)  
  }  
}
```

Resposta

```
{  
  "cmd": "prg_rtc_w",  
  "sts": true  
}
```

Escrita (Central para o equipamento)

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	Payload (X Byte)	CRC 16 (2 Bytes)
0xA5	0x00: Central -> ALX	0x04	0x00	0x06	UInt32_t timestamp UInt8_t utc UInt8_t ntp	0xFFFF

Resposta (Equipamento para o Central)

Start Of Frame (1 Byte)	Direção (1 Byte)	CMD_ID (2 Bytes)	Fragmentação (1 Byte)	Tamanho Payload (2 Bytes)	Payload (X Byte)	CRC 16 (2 Bytes)
0xA5	0x01: ALX -> Central	0x04	0x00	0x1F	UInt8_t resultado Char detalhe [30]	0xFFFF

Configuração dos Drivers:

O procedimento para o envio de drivers ao ALX via Bluetooth mantém o cabeçalho e a validação padrão do protocolo, apresentando alterações apenas na estrutura interna do campo *Payload*.

Diretrizes para o envio de drivers:

- Transmissão Singular:** O comando permite o envio de apenas um (1) driver por execução.
- Múltiplos Drivers:** Para transferir mais de um driver, o aplicativo central deve repetir o envio deste comando de forma sequencial, iterando sobre cada driver necessário.

Formatação exigida para o campo Payload:

